Avoiding Vendor Lock-In in AI Procurement for the DoW

Strategies for ML Platform Independence in Naval AI Systems

Sam Bright · William Emeny · Alan Jaeger · Adam Larson · JP Lueck · Michael Soltys

GBL Systems Corp. · NSWC Port Hueneme Division

What is Vendor Lock-In?

Defining the Problem

- Switching costs become so high that an organization is effectively **trapped** with a single provider
- We define it across **four dimensions**: platform, data, model, and expertise dependencies

Impact on the DoW

- **Technological sovereignty** – losing control of critical AI capabilities
- **Competition** – sole-source lets vendors increase prices
- **Operational flexibility** – inability to adapt as threats evolve
- **Innovation access** – locked out of better solutions

> *"What begins as a useful tool can become an iron cage of constraint." – Max Weber*

The $112 Million Lesson - USDA Case Study (2021)

What Happened

Paid **$112 million more** for Microsoft Office than Google Workspace – specifically to avoid even *higher* switching costs.

Migrating would have required:

- Retraining thousands of employees
- Converting years of templates and workflows
- Rebuilding integrations with dozens of internal systems

Why This Matters for Defense AI

- This was commodity office software
- Not mission-critical AI

For defense AI – classified data, mission-critical uptime, proprietary model weights, security requirements – switching barriers are **orders of magnitude** greater.

> *If commodity software costs $112M to escape, what does defense AI lock-in cost?*

Historical Context: Lock-In Through the Decades

How Has Lock-In Presented Itself in Past Architectures?

- **1960s–80s: Mainframe era** – IBM's proprietary hardware and software created total dependency; migrating off a mainframe meant rebuilding everything from scratch
- **1990s: Client-server** – Microsoft and Oracle established dominance through proprietary protocols, file formats, and database engines that resisted interoperability
- **2000s: Enterprise software** – ERP and CRM vendors (SAP, Salesforce) created lock-in through deeply customized implementations and proprietary data models
- **2010s: Cloud computing** – Lock-in came not from generic services (Linux VMs, object storage) but from *managed services* like AWS Lambda and Azure Functions – convenient, but dangerous to build around

> *Each technology wave created new opportunities for vendors to establish proprietary dominance*

What Does Lock-In Look Like Today?

Key Vulnerabilities in Modern AI Architectures

Deeper Than Before

- AI creates dependencies beyond software interfaces – into **intellectual property and algorithmic competence**
- Models trained on proprietary platforms can become **inseparable** from their training environments
- Vendor-specific optimization means performance degrades on migration

The Cloud AI Ecosystem Trap

- **AWS**: Bedrock, SageMaker, Rekognition
- **Azure**: OpenAI Service, Azure ML, Cognitive Services
- **Google**: Vertex AI, AutoML, specialized NLP APIs
- Each integrates seamlessly *within* its ecosystem while creating friction for cross-platform migration

Unlike previous waves, AI lock-in extends into models themselves – not just infrastructure

Four Types of AI Vendor Lock-In

Platform Lock-In

Training, inference, and deployment tied to a specific cloud ecosystem (Azure OpenAI, AWS Bedrock/SageMaker, Google Vertex AI)

Data Lock-In

Training datasets and preprocessing pipelines optimized for vendor-specific formats and storage systems

Model Lock-In

Models dependent on proprietary frameworks, custom accelerators, or vendor-specific optimization libraries

Expertise Lock-In

Teams specialized in vendor-specific tools and APIs; organizational inertia reinforces the relationship

These four types compound each other – together, they create lock-in

Defense-Specific Risks

Why the DoW Is Uniquely Vulnerable

- **Classification constraints** limit ability to evaluate alternatives
- **Clearance requirements** restrict the vendor pool
- **Mission-critical reliability** makes organizations risk-averse to platform changes
- **Long procurement cycles** (2–5 years) lock in choices made years earlier
- **Trust inertia** – users develop confidence in a model's specific behaviors and "quirks" that doesn't transfer to new systems

DoW Open Source Software memo (Jan 2022): Lock-in can occur *even with open source* through expertise dependencies and specialized implementations.

Good news: CDAO has already proven vendor-agnostic approaches work at DoW scale

Policy Mandate: Sec Hegseth Acquisition Reforms

November 7, 2025 – Secretary Hegseth, National War College

Four Core Contractor Requirements

1. Accept risk and invest in capacity
2. Embrace commercial "85% solutions"
3. **Supply chain resilience** ← DUAL-SOURCE MANDATE
4. Performance-based contracting

Structural Changes

- Portfolio Acquisition Executives (PAEs) replace Program Executive Offices
- Compensation tied to delivery speed and competition
- "Speed to delivery" as organizing principle
- Lock-in avoidance: *best practice* → **mandatory policy**

> *"Maintain at least two qualified sources for critical program content" – Secretary Hegseth*

MS

Technical Strategies for Platform Independence

Containerization

- OCI-compliant containers + Kubernetes
- Portable ML environments
- Kubeflow for vendor-neutral workflows
- Standardized GPU management

Open Standards

- **ONNX** – universal model format (1,700+ tools)
- **MLflow** – experiment tracking (800+ contributors)
- **Apache Iceberg** – vendor-neutral data lakes

Infrastructure-as-Code

- **Terraform** – deploy to AWS, Azure, GCP, or on-prem
- **JWCC** – multi-vendor cloud within DoW
- **REST + OpenAPI** – swap backends freely

These are mature, production-ready technologies – not research projects

MS

Procurement Strategies

Contracts Must Enable Technical Independence

Modular Contracting

- Break AI projects into interoperable increments
- Multiple vendors compete for each module
- Supported by DoW Software Acquisition Pathway

Government Data Rights

- Retain unlimited rights to: training data, model weights, source code
- Require SBOM
- Distinguish background IP from foreground IP

Performance-Based

- Define success by accuracy, availability, effectiveness
- Include portability in SLAs
- Reward vendor-independent implementations

Technical strategies only work if contracts support them

MS

Five Things to Require in Your Next AI Solicitation

1. **Containerized Delivery** – OCI-compliant containers on any Kubernetes cluster, not tied to one cloud
2. **Open Model Formats** – ONNX or equivalent, so models run on alternative inference engines
3. **Government Data Rights** – Unlimited rights to training data, model weights, and source code
4. **Infrastructure-as-Code** – Terraform or equivalent, reproducible on any provider
5. **Demonstrated Portability** – Must deploy and pass tests on at least two environments

MS

Lessons from DoW Programs

Vendor Independence Works at Operational Scale

CDAO GIDE

- Vendor-agnostic data integration across **50+ sources**
- Availability **>99.5%** across multiple vendor environments
- Tactical decision-making latencies are met

Army Enterprise Cloud

- Multi-cloud across AWS, Azure, and Google Cloud
- Cloud-agnostic management layers
- Improved resilience and cost optimization

Navy AI Modernization

- **"Buy operational outcomes, not AI"**
- Performance-based contracts with portability
- Robust follow-on competition maintained

MS

Closing

The Risk

$112M+ switching costs Four compounding lock-in types Defense amplifies every barrier

The Mandate

Hegseth dual-source policy PAEs incentivized for competition No longer optional

The Solution

Five checklist items Proven at DoW scale Mature technologies

Vendor independence is achievable – and now mandatory

The question isn't whether to pursue vendor independence – it's how fast you can get there.